Source-Code Evidence for the Real-Time AI Governance Audit Pipeline

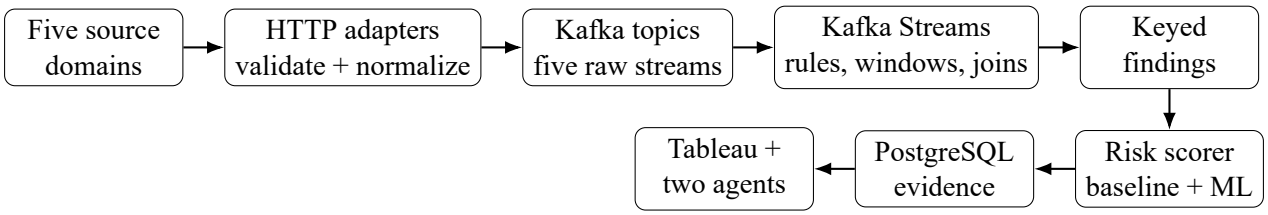
Aditya Ranjan Arindam Gupta Gnanendra Naidu N

November 2026

Abstract

This document records the implemented pipeline using screenshots generated directly from the repository source files. Each screenshot is tied to a source path and represents the corresponding stage described in the project report. The sequence follows the exact flow: producer adapters, Kafka topics and configuration, Kafka Streams processing, governance rules, risk scoring, PostgreSQL evidence, agent investigation and review, evaluation and recovery, and reporting.

Pipeline represented in the report



The screenshots are not mock-ups or reconstructed examples. They are rendered views of source excerpts read from the working repository. The source path is printed in each image header so that every implementation claim can be traced to a file.

1 1. Producer adapters and event normalization

The pipeline starts with five producer domains: Keycloak identity, Kubernetes audit, Airflow/OpenLineage pipeline activity, model registry activity, and LLM gateway activity. The shared mapper layer converts source-specific payloads into the canonical governance-event contract while retaining source identifiers and evidence metadata.

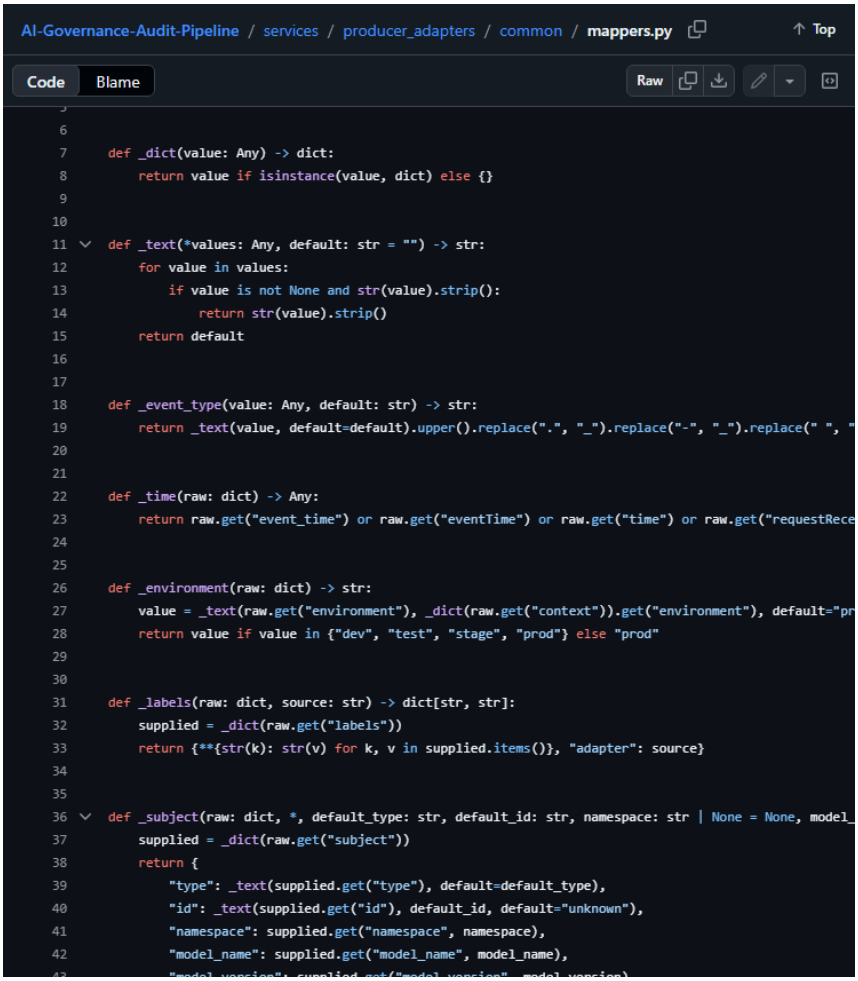


Figure 1: Actual GitHub source page, top segment, for the producer adapter mapping layer.



```
            "data_classification": supplied.get("data_classification"),
        }


def keycloak(raw: dict) -> dict:
    auth = _dict(raw.get("authDetails"))
    details = _dict(raw.get("details"))
    supplied_actor = _dict(raw.get("actor"))
    source_type = _event_type(raw.get("event_type") or raw.get("type") or raw.get("operationType"),
    aliases = {"LOGIN_ERROR": "LOGIN_FAILURE", "LOGIN": "LOGIN_SUCCESS", "ROLE_MAPPING_CREATE": "RO
    event_type = aliases.get(source_type, source_type)
    actor_id = _text(supplied_actor.get("id"), auth.get("userId"), raw.get("userId"), details.get("
    attributes = {**details, **_dict(raw.get("attributes"))}
    return {
        "source_id": raw.get("id") or raw.get("event_id"),
        "event_time": _time(raw),
        "event_type": event_type,
        "actor": {
            "id": actor_id,
            "type": supplied_actor.get("type", "USER"),
            "roles": supplied_actor.get("roles", raw.get("roles", [])),
            "ip": supplied_actor.get("ip", auth.get("ipAddress") or raw.get("ipAddress")),
            "namespace": supplied_actor.get("namespace"),
        },
        "subject": _subject(raw, default_type="IDENTITY", default_id=_text(raw.get("resourcePath"),
        "action": _text(raw.get("action"), event_type.lower()),
        "outcome": raw.get("outcome") or ("FAILURE" if raw.get("error") or event_type == "LOGIN_FAI
        "environment": _environment(raw),
        "correlation_id": _text(raw.get("correlation_id"), raw.get("sessionId"), raw.get("id"), def
        "trace_id": raw.get("trace_id"),
        "attributes": attributes,
        "labels": _labels(raw, "keycloak"),
    }


def kubernetes_audit(raw: dict) -> dict:
    user = _dict(raw.get("user"))
    object_ref = _dict(raw.get("objectRef"))
    request_object = _dict(raw.get("requestObject"))
    metadata = _dict(request_object.get("metadata"))
    annotations = _dict(metadata.get("annotations"))
    supplied = _dict(raw.get("attributes"))
```

Figure 2: Actual GitHub source page, middle segment, for the producer adapter mapping layer.



```
        verb = _text(raw.get("verb"), raw.get("action"), default="unknown").lower()
        resource = _text(object_ref.get("resource"), default="resource").lower()
        if raw.get("event_type"):
            event_type = _event_type(raw["event_type"], "KUBERNETES_AUDIT")
        elif verb in {"create", "update", "patch"} and resource == "clusterrolebindings":
            event_type = "CLUSTER_ROLE_BINDING"
        elif verb in {"get", "list", "watch"} and resource == "secrets":
            event_type = "SECRET_ACCESS"
        elif verb == "create" and resource == "pods":
            event_type = "POD_CREATE"
        else:
            event_type = _event_type(f"K8S_{verb}_{resource}", "KUBERNETES_AUDIT")

        spec = _dict(request_object.get("spec"))
        containers = list(spec.get("containers") or []) + list(spec.get("initContainers") or [])
        privileged = any(bool(_dict(_dict(c).get("securityContext")).get("privileged")) for c in contai
        namespace = object_ref.get("namespace") or metadata.get("namespace")
        role_ref = _dict(request_object.get("roleRef"))
        attributes = {
            "cluster_role": _text(supplied.get("cluster_role"), role_ref.get("name"), object_ref.get("n
            "change_approved": bool(supplied.get("change_approved", annotations.get("governance.example
            "cross_namespace": bool(supplied.get("cross_namespace", False)),
            "privileged": bool(supplied.get("privileged", privileged)),
            "image_approved": bool(supplied.get("image_approved", annotations.get("governance.example/i
            **supplied,
        }
        actor_id = _text(_dict(raw.get("actor")).get("id"), user.get("username"), default="unknown")
        return {
            "source_id": raw.get("auditID") or raw.get("event_id"),
            "event_time": _time(raw) or raw.get("stageTimestamp"),
            "event_type": event_type,
            "actor": {
                "id": actor_id,
                "type": "SERVICE" if actor_id.startswith("system:serviceaccount:") else "USER",
                "roles": user.get("groups", []),
                "ip": (raw.get("sourceIPs") or [None])[0],
                "namespace": namespace,
            },
            "subject": _subject(raw, default_type=resource.upper(), default_id=_text(object_ref.get("na
            "action": verb,
            "outcome": "FAILURE" if int(_dict(raw.get("responseStatus")).get("code", 200)) >= 400 else
            "environment": _environment(raw),
```

Figure 3: Actual GitHub source page, final segment, for the producer adapter mapping layer.



```
            "trace_id": raw.get("trace_id"),
            "attributes": attributes,
            "labels": _labels(raw, "kubernetes-audit"),
        }


def airflow_openlineage(raw: dict) -> dict:
    run = _dict(raw.get("run"))
    job = _dict(raw.get("job"))
    run_facets = _dict(run.get("facets"))
    job_facets = _dict(job.get("facets"))
    governance = {**_dict(job_facets.get("governance")), **_dict(run_facets.get("governance")), **_
    event_type = _event_type(raw.get("event_type") or governance.pop("event_type", None) or raw.get
    job_name = _text(job.get("name"), raw.get("dag_id"), default="unknown-job")
    namespace = _text(job.get("namespace"), raw.get("namespace"), default="airflow")
    actor_id = _text(_dict(raw.get("actor")).get("id"), raw.get("service_account"), default="airflo
    return {
        "source_id": run.get("runId") or raw.get("run_id") or raw.get("event_id"),
        "event_time": _time(raw),
        "event_type": event_type,
        "actor": {"id": actor_id, "type": "SERVICE", "roles": ["pipeline"], "ip": None, "namespace"
        "subject": _subject(raw, default_type="PIPELINE_JOB", default_id=job_name, namespace=namesp
        "action": _text(raw.get("action"), event_type.lower()),
        "outcome": "FAILURE" if event_type in {"FAIL", "FAILED"} else "SUCCESS",
        "environment": _environment(raw),
        "correlation_id": _text(raw.get("correlation_id"), run.get("runId"), default=f"pipeline:{jo
        "trace_id": raw.get("trace_id"),
        "attributes": governance,
        "labels": _labels(raw, "airflow-openlineage"),
    }


def model_registry(raw: dict) -> dict:
    model = _dict(raw.get("model"))
    supplied_subject = _dict(raw.get("subject"))
    name = _text(supplied_subject.get("model_name"), model.get("name"), raw.get("model_name"), defa
    version = _text(supplied_subject.get("model_version"), model.get("version"), raw.get("model_ver
    source_type = _event_type(raw.get("event_type") or raw.get("action") or raw.get("type"), "MODEL
    aliases = {
        "PROMOTED": "MODEL_PROMOTED", "MODEL_PROMOTION": "MODEL_PROMOTED",
        "APPROVED": "APPROVAL_GRANTED", "EVALUATION_COMPLETED": "MODEL_EVALUATION_COMPLETED",
        "ARTIFACT_CREATED": "ARTIFACT_REGISTERED",
```

Figure 4: Actual GitHub source page, continued producer adapter mapping.



```
    }
    event_type = aliases.get(source_type, source_type)
    actor = _dict(raw.get("actor"))
    return {
        "source_id": raw.get("id") or raw.get("event_id"),
        "event_time": _time(raw),
        "event_type": event_type,
        "actor": {
            "id": _text(actor.get("id"), raw.get("user"), default="model-registry"),
            "type": actor.get("type", "SERVICE"), "roles": actor.get("roles", []),
```

Figure 5: Actual GitHub source page, continued producer adapter mapping.



Figure 6: Actual GitHub source page, final producer adapter mapping segment.

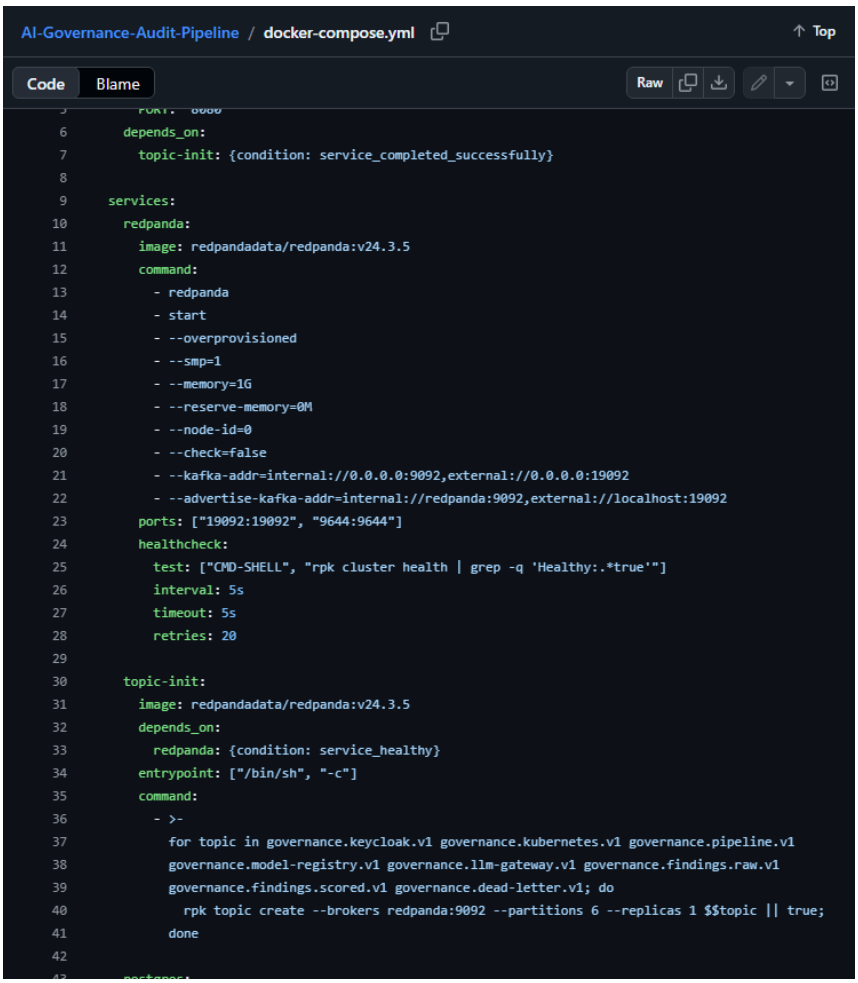


Figure 7: Actual GitHub source page, top segment, for Kafka configuration.



```
  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: governance
      POSTGRES_USER: governance
      POSTGRES_PASSWORD: governance
    ports: ["127.0.0.1:5432:5432"]
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./db/001_schema.sql:/docker-entrypoint-initdb.d/001_schema.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U governance -d governance"]
      interval: 5s
      timeout: 5s
      retries: 20

  keycloak-adapter:
    <<: *producer-adapter
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: keycloak}
    ports: ["8101:8080"]

  kubernetes-audit-adapter:
    <<: *producer-adapter
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: kubernetes_audit}
    ports: ["8102:8080"]

  airflow-openlineage-adapter:
    <<: *producer-adapter
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: airflow_openlineage}
    ports: ["8103:8080"]

  model-registry-adapter:
    <<: *producer-adapter
```

Figure 8: Actual GitHub source page, middle segment, for Kafka configuration.



```
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: model_registry}
    ports: ["8104:8080"]

  llm-gateway-adapter:
    <<: *producer-adapter
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: llm_gateway}
    ports: ["8105:8080"]

  stream-processor:
    build:
      context: .
      dockerfile: stream-processor/Dockerfile
    environment:
      KAFKA_BOOTSTRAP_SERVERS: redpanda:9092
      KAFKA_SECURITY_PROTOCOL: PLAINTEXT
      KAFKA_INTERNAL_REPLICATION_FACTOR: "1"
      KAFKA_STREAM_THREADS: "2"
    depends_on:
      topic-init: {condition: service_completed_successfully}

  risk-scorer:
    build:
      context: .
      dockerfile: services/risk-scorer/Dockerfile
    environment:
      KAFKA_BOOTSTRAP_SERVERS: redpanda:9092
      KAFKA_SECURITY_PROTOCOL: PLAINTEXT
      RISK_MODEL_MIN_SAMPLES: ${RISK_MODEL_MIN_SAMPLES:-32}
      RISK_MODEL_RETRAIN_EVERY: ${RISK_MODEL_RETRAIN_EVERY:-25}
    depends_on:
      topic-init: {condition: service_completed_successfully}

  postgres-sink:
    build:
      context: .
      dockerfile: services/postgres-sink/Dockerfile
```

Figure 9: Actual GitHub source page, final segment, for Kafka configuration.



```
      environment:
        KAFKA_BOOTSTRAP_SERVERS: redpanda:9092
        KAFKA_SECURITY_PROTOCOL: PLAINTEXT
        POSTGRES_DSN: postgresql://governance:governance@postgres:5432/governance
      depends_on:
        topic-init: {condition: service_completed_successfully}
        postgres: {condition: service_healthy}

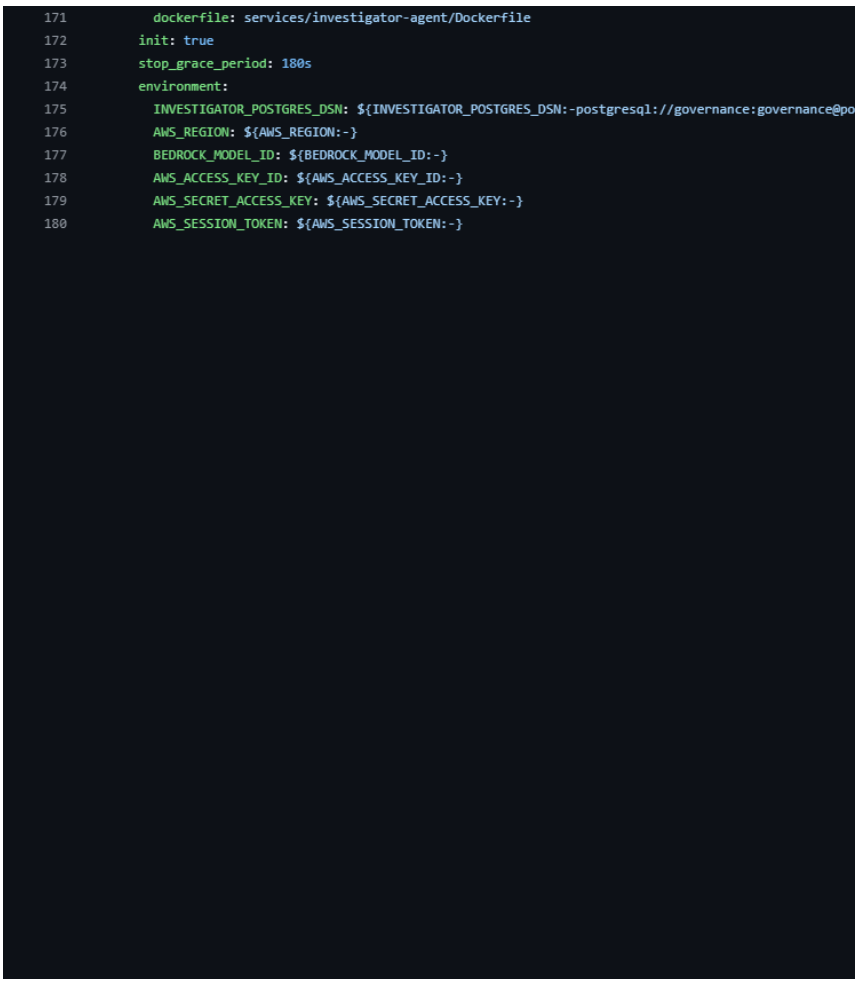
  evaluator:
    profiles: ["evaluation"]
    build:
      context: .
      dockerfile: evaluation/Dockerfile
    environment:
      KAFKA_BOOTSTRAP_SERVERS: redpanda:9092
      POSTGRES_DSN: postgresql://governance:governance@postgres:5432/governance
    depends_on:
      topic-init: {condition: service_completed_successfully}
      postgres: {condition: service_healthy}

  producer-adapter-tests:
    profiles: ["test"]
    build:
      context: .
      dockerfile: services/producer_adapters/Dockerfile
      args: {ADAPTER: keycloak}
    command: ["python", "-m", "unittest", "discover", "-s", "tests", "-v"]

  investigator-db-init:
    profiles: ["agents"]
    build:
      context: .
      dockerfile: services/investigator-agent/Dockerfile
    command: ["migrate"]
    environment:
      INVESTIGATOR_POSTGRES_DSN: ${INVESTIGATOR_ADMIN_DSN:-postgresql://governance:governance@postg
    depends_on:
      postgres: {condition: service_healthy}

  investigator-agent:
    profiles: ["agents"]
    build:
      context: .
```

Figure 10: Actual GitHub source page, continued Kafka configuration.



```
171           dockerfile: services/investigator-agent/Dockerfile
172         init: true
173         stop_grace_period: 180s
174         environment:
175           INVESTIGATOR_POSTGRES_DSN: ${INVESTIGATOR_POSTGRES_DSN:-postgresql://governance:governance@po
176           AWS_REGION: ${AWS_REGION:-}
177           BEDROCK_MODEL_ID: ${BEDROCK_MODEL_ID:-}
178           AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID:-}
179           AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY:-}
180           AWS_SESSION_TOKEN: ${AWS_SESSION_TOKEN:-}
```

Figure 11: Actual GitHub source page, continued Kafka configuration.



Figure 12: Actual GitHub source page, final Kafka configuration segment.

2 2. Kafka topics and stream processing

The normalized events are published to dedicated raw topics. The stream processor consumes those topics using Kafka Streams. The implementation uses event timestamps, state stores, windows, table-table joins, stream-table joins, keyed findings, and JSON serialization. The screenshot below is the actual topology source used to connect those operations.

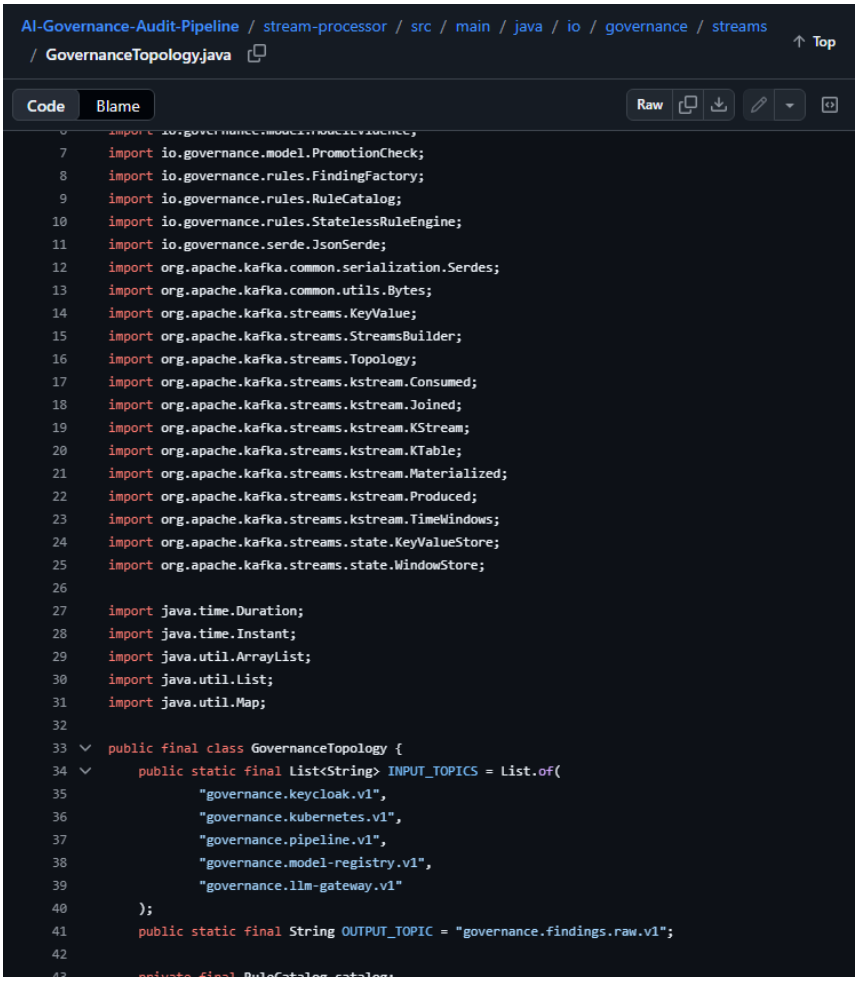
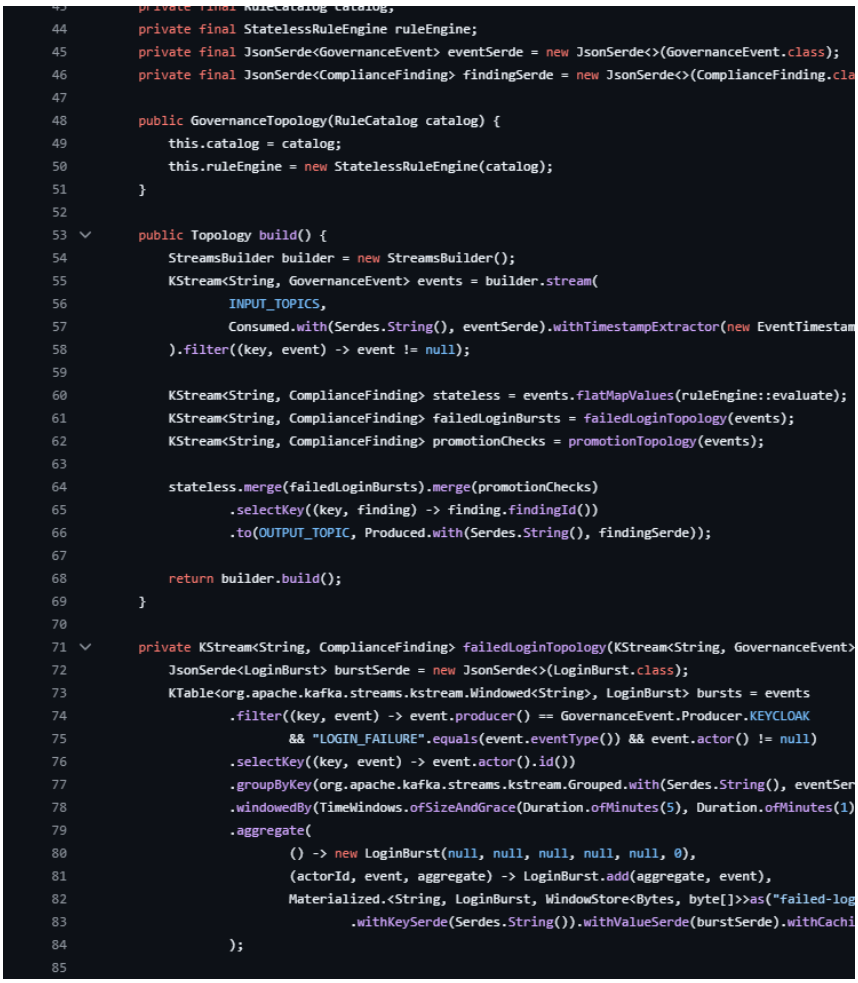


Figure 13: Actual GitHub source page, top segment, for the Kafka Streams topology.



```
    private final StatelessRuleEngine ruleEngine;
    private final JsonSerde<GovernanceEvent> eventSerde = new JsonSerde<>(GovernanceEvent.class);
    private final JsonSerde<ComplianceFinding> findingSerde = new JsonSerde<>(ComplianceFinding.cla

    public GovernanceTopology(RuleCatalog catalog) {
        this.catalog = catalog;
        this.ruleEngine = new StatelessRuleEngine(catalog);
    }

    public Topology build() {
        StreamsBuilder builder = new StreamsBuilder();
        KStream<String, GovernanceEvent> events = builder.stream(
                INPUT_TOPICS,
                Consumed.with(Serdes.String(), eventSerde).withTimestampExtractor(new EventTimestam
        ).filter((key, event) -> event != null);

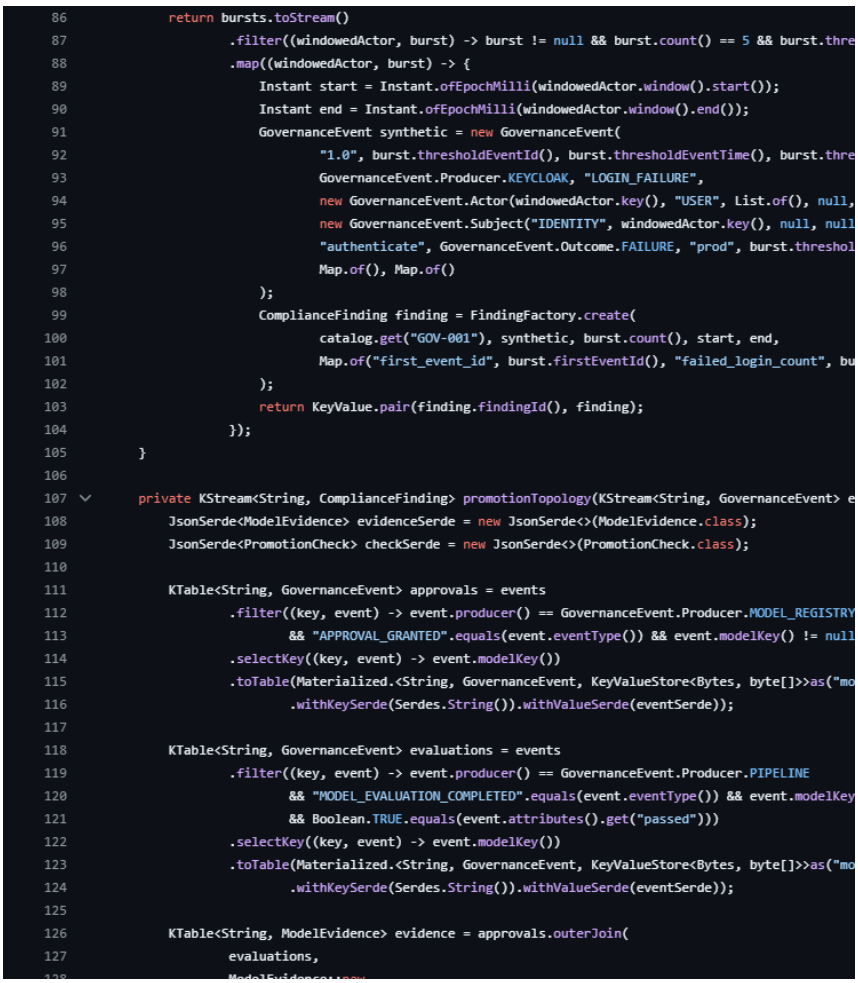
        KStream<String, ComplianceFinding> stateless = events.flatMapValues(ruleEngine::evaluate);
        KStream<String, ComplianceFinding> failedLoginBursts = failedLoginTopology(events);
        KStream<String, ComplianceFinding> promotionChecks = promotionTopology(events);

        stateless.merge(failedLoginBursts).merge(promotionChecks)
                .selectKey((key, finding) -> finding.findingId())
                .to(OUTPUT_TOPIC, Produced.with(Serdes.String(), findingSerde));

        return builder.build();
    }

    private KStream<String, ComplianceFinding> failedLoginTopology(KStream<String, GovernanceEvent>
        JsonSerde<LoginBurst> burstSerde = new JsonSerde<>(LoginBurst.class);
        KTable<org.apache.kafka.streams.kstream.Windowed<String>, LoginBurst> bursts = events
                .filter((key, event) -> event.producer() == GovernanceEvent.Producer.KEYCLOAK
                        && "LOGIN_FAILURE".equals(event.eventType()) && event.actor() != null)
                .selectKey((key, event) -> event.actor().id())
                .groupByKey(org.apache.kafka.streams.kstream.Grouped.with(Serdes.String(), eventSer
                .windowedBy(TimeWindows.ofSizeAndGrace(Duration.ofMinutes(5), Duration.ofMinutes(1)
                .aggregate(
                        () -> new LoginBurst(null, null, null, null, null, 0),
                        (actorId, event, aggregate) -> LoginBurst.add(aggregate, event),
                        Materialized.<String, LoginBurst, WindowStore<Bytes, byte[]>>as("failed-log
                                .withKeySerde(Serdes.String()).withValueSerde(burstSerde).withCachi
                );
```

Figure 14: Actual GitHub source page, final segment, for rules, windows, joins, and finding emission.



```
        return bursts.toStream()
                .filter((windowedActor, burst) -> burst != null && burst.count() == 5 && burst.thre
                .map((windowedActor, burst) -> {
                    Instant start = Instant.ofEpochMilli(windowedActor.window().start());
                    Instant end = Instant.ofEpochMilli(windowedActor.window().end());
                    GovernanceEvent synthetic = new GovernanceEvent(
                            "1.0", burst.thresholdEventId(), burst.thresholdEventTime(), burst.thre
                            GovernanceEvent.Producer.KEYCLOAK, "LOGIN_FAILURE",
                            new GovernanceEvent.Actor(windowedActor.key(), "USER", List.of(), null,
                            new GovernanceEvent.Subject("IDENTITY", windowedActor.key(), null, null
                            "authenticate", GovernanceEvent.Outcome.FAILURE, "prod", burst.threshol
                            Map.of(), Map.of()
                    );
                    ComplianceFinding finding = FindingFactory.create(
                            catalog.get("GOV-001"), synthetic, burst.count(), start, end,
                            Map.of("first_event_id", burst.firstEventId(), "failed_login_count", bu
                    );
                    return KeyValue.pair(finding.findingId(), finding);
                });
    }

    private KStream<String, ComplianceFinding> promotionTopology(KStream<String, GovernanceEvent> e
        JsonSerde<ModelEvidence> evidenceSerde = new JsonSerde<>(ModelEvidence.class);
        JsonSerde<PromotionCheck> checkSerde = new JsonSerde<>(PromotionCheck.class);

        KTable<String, GovernanceEvent> approvals = events
                .filter((key, event) -> event.producer() == GovernanceEvent.Producer.MODEL_REGISTRY
                        && "APPROVAL_GRANTED".equals(event.eventType()) && event.modelKey() != null
                .selectKey((key, event) -> event.modelKey())
                .toTable(Materialized.<String, GovernanceEvent, KeyValueStore<Bytes, byte[]>>as("mo
                        .withKeySerde(Serdes.String()).withValueSerde(eventSerde));

        KTable<String, GovernanceEvent> evaluations = events
                .filter((key, event) -> event.producer() == GovernanceEvent.Producer.PIPELINE
                        && "MODEL_EVALUATION_COMPLETED".equals(event.eventType()) && event.modelKey
                        && Boolean.TRUE.equals(event.attributes().get("passed")))
                .selectKey((key, event) -> event.modelKey())
                .toTable(Materialized.<String, GovernanceEvent, KeyValueStore<Bytes, byte[]>>as("mo
                        .withKeySerde(Serdes.String()).withValueSerde(eventSerde));

        KTable<String, ModelEvidence> evidence = approvals.outerJoin(
                evaluations,
```

Figure 15: Actual GitHub source page, continued stream topology.

```
                    ModelEvidence::new,
                    Materialized.<String, ModelEvidence, KeyValueStore<Bytes, byte[]>>as("model-governa
                            .withKeySerde(Serdes.String()).withValueSerde(evidenceSerde)
            );

            KStream<String, PromotionCheck> checks = events
                    .filter((key, event) -> event.producer() == GovernanceEvent.Producer.MODEL_REGISTRY
                            && "MODEL_PROMOTED".equals(event.eventType()) && "prod".equals(event.enviro
                            && event.modelKey() != null)
                    .selectKey((key, event) -> event.modelKey())
                    .leftJoin(evidence, PromotionCheck::new, Joined.with(Serdes.String(), eventSerde, e

            return checks.flatMapValues(check -> {
                List<ComplianceFinding> findings = new ArrayList<>();
                ModelEvidence modelEvidence = check.evidence();
                if (modelEvidence == null || modelEvidence.approval() == null) {
                    findings.add(FindingFactory.create(catalog.get("GOV-011"), check.promotion(), 1, nu
                            Map.of("model_key", check.promotion().modelKey(), "approval_present", false
                }
                if (modelEvidence == null || modelEvidence.evaluation() == null) {
                    findings.add(FindingFactory.create(catalog.get("GOV-012"), check.promotion(), 1, nu
                            Map.of("model_key", check.promotion().modelKey(), "evaluation_present", fal
                }
                return findings;
            });
        }
    }
```

Figure 16: Actual GitHub source page, final stream topology segment.

3 3. Governance rule catalogue

The rule catalogue is configuration-driven. It associates executable controls with severity and governance-framework metadata. Rule identity is preserved in every finding so that a reviewer can connect a result to the exact configured condition.

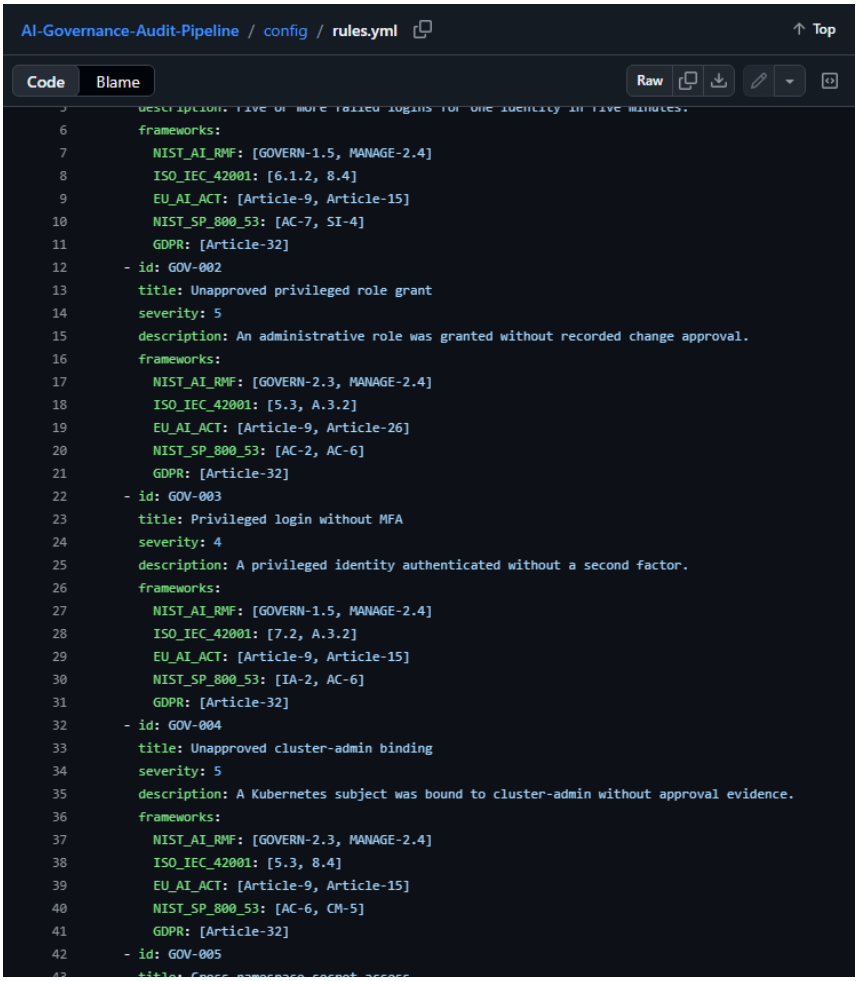


Figure 17: Actual GitHub source page, top segment, for the governance rule catalogue.

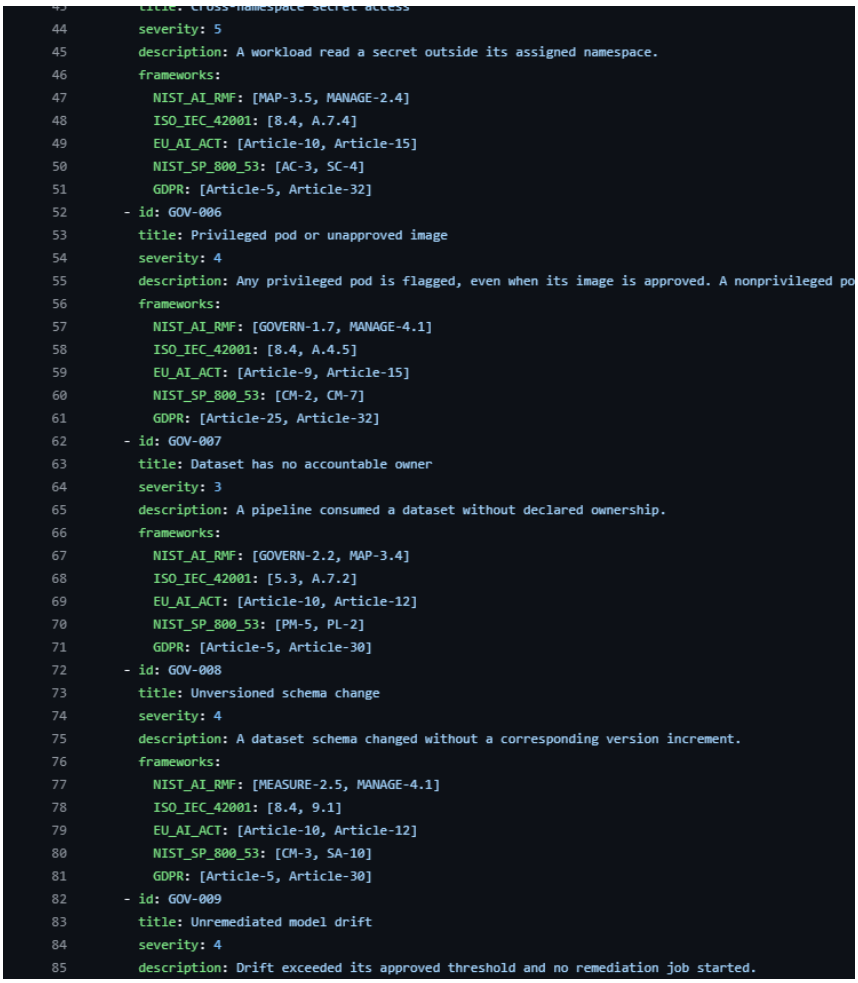
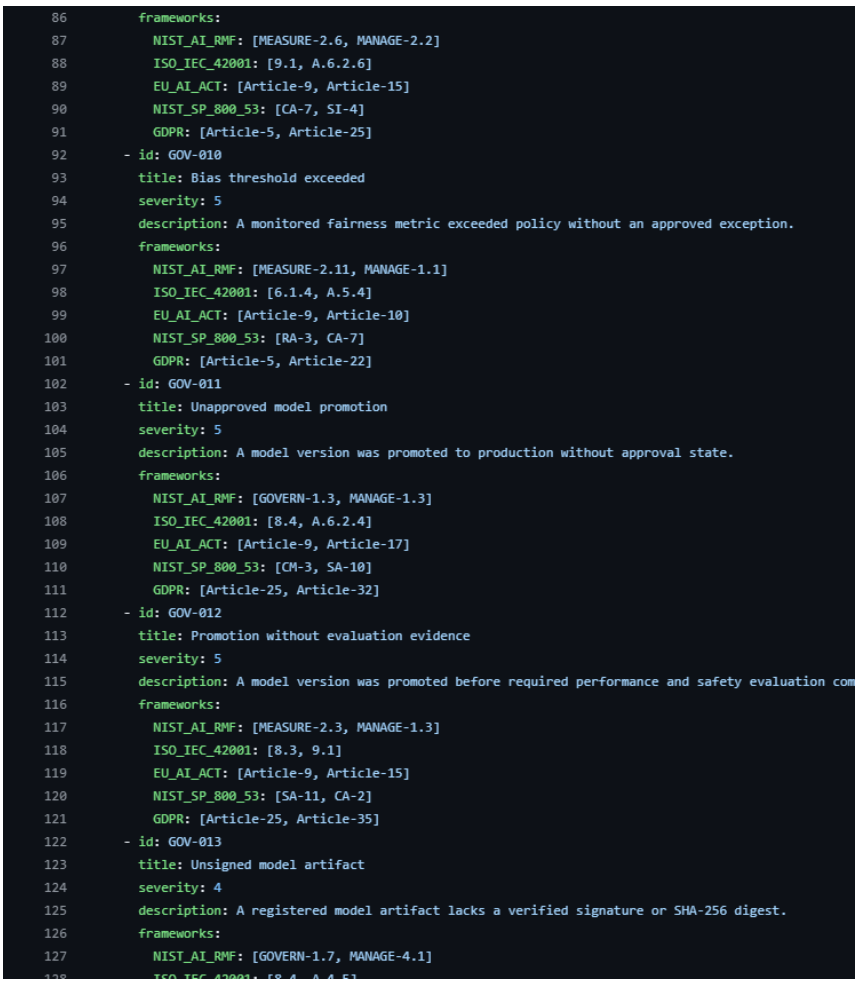


Figure 18: Actual GitHub source page, final segment, for the governance rule catalogue.



```
    frameworks:
      NIST_AI_RMF: [MEASURE-2.6, MANAGE-2.2]
      ISO_IEC_42001: [9.1, A.6.2.6]
      EU_AI_ACT: [Article-9, Article-15]
      NIST_SP_800_53: [CA-7, SI-4]
      GDPR: [Article-5, Article-25]
  - id: GOV-010
    title: Bias threshold exceeded
    severity: 5
    description: A monitored fairness metric exceeded policy without an approved exception.
    frameworks:
      NIST_AI_RMF: [MEASURE-2.11, MANAGE-1.1]
      ISO_IEC_42001: [6.1.4, A.5.4]
      EU_AI_ACT: [Article-9, Article-10]
      NIST_SP_800_53: [RA-3, CA-7]
      GDPR: [Article-5, Article-22]
  - id: GOV-011
    title: Unapproved model promotion
    severity: 5
    description: A model version was promoted to production without approval state.
    frameworks:
      NIST_AI_RMF: [GOVERN-1.3, MANAGE-1.3]
      ISO_IEC_42001: [8.4, A.6.2.4]
      EU_AI_ACT: [Article-9, Article-17]
      NIST_SP_800_53: [CM-3, SA-10]
      GDPR: [Article-25, Article-32]
  - id: GOV-012
    title: Promotion without evaluation evidence
    severity: 5
    description: A model version was promoted before required performance and safety evaluation com
    frameworks:
      NIST_AI_RMF: [MEASURE-2.3, MANAGE-1.3]
      ISO_IEC_42001: [8.3, 9.1]
      EU_AI_ACT: [Article-9, Article-15]
      NIST_SP_800_53: [SA-11, CA-2]
      GDPR: [Article-25, Article-35]
  - id: GOV-013
    title: Unsigned model artifact
    severity: 4
    description: A registered model artifact lacks a verified signature or SHA-256 digest.
    frameworks:
      NIST_AI_RMF: [GOVERN-1.7, MANAGE-4.1]
```

Figure 19: Actual GitHub source page, continued governance rule catalogue.

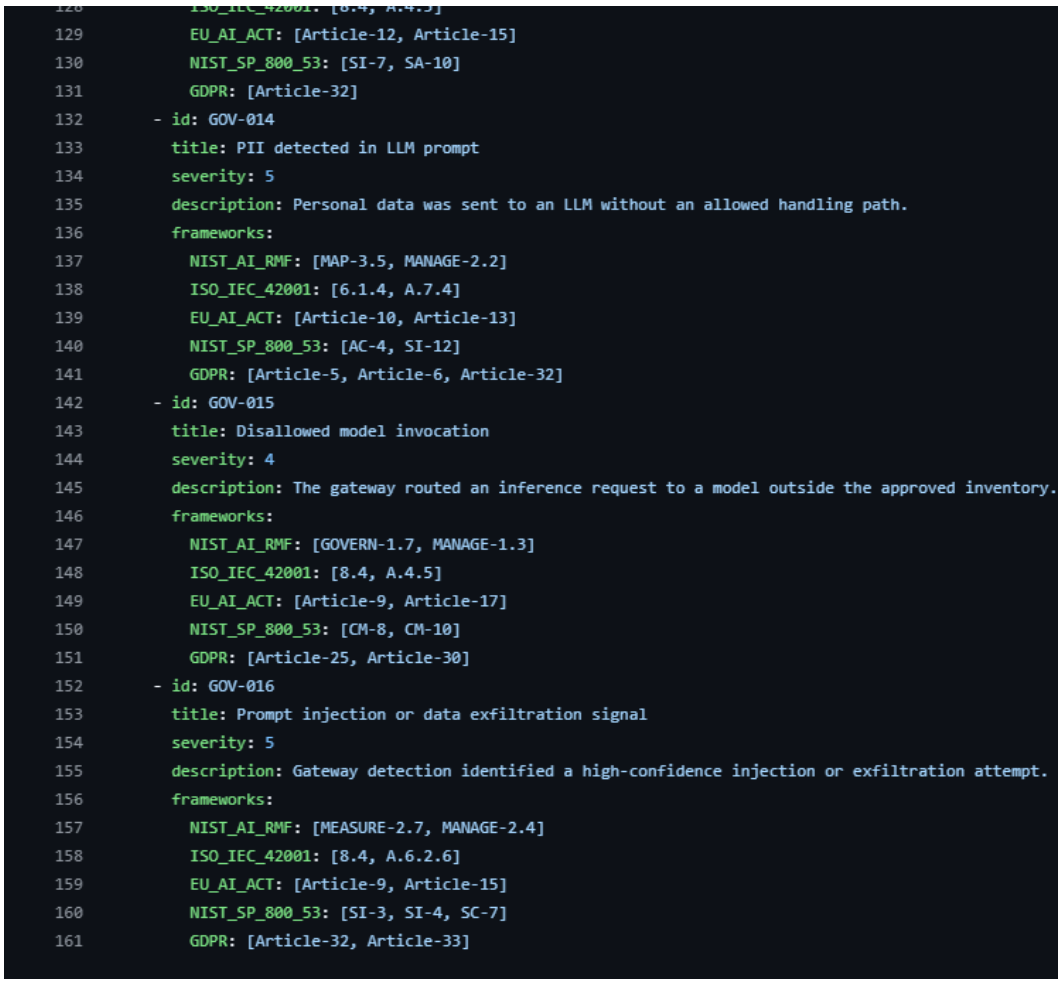


Figure 20: Actual GitHub source page, final governance rule catalogue segment.

4 4. Risk scoring and durable evidence

After stream processing, findings are scored and persisted. The risk service keeps the deterministic baseline visible and adds an online Isolation Forest comparison once sufficient history is available. The PostgreSQL sink performs idempotent writes for events, findings, evidence, and scores.

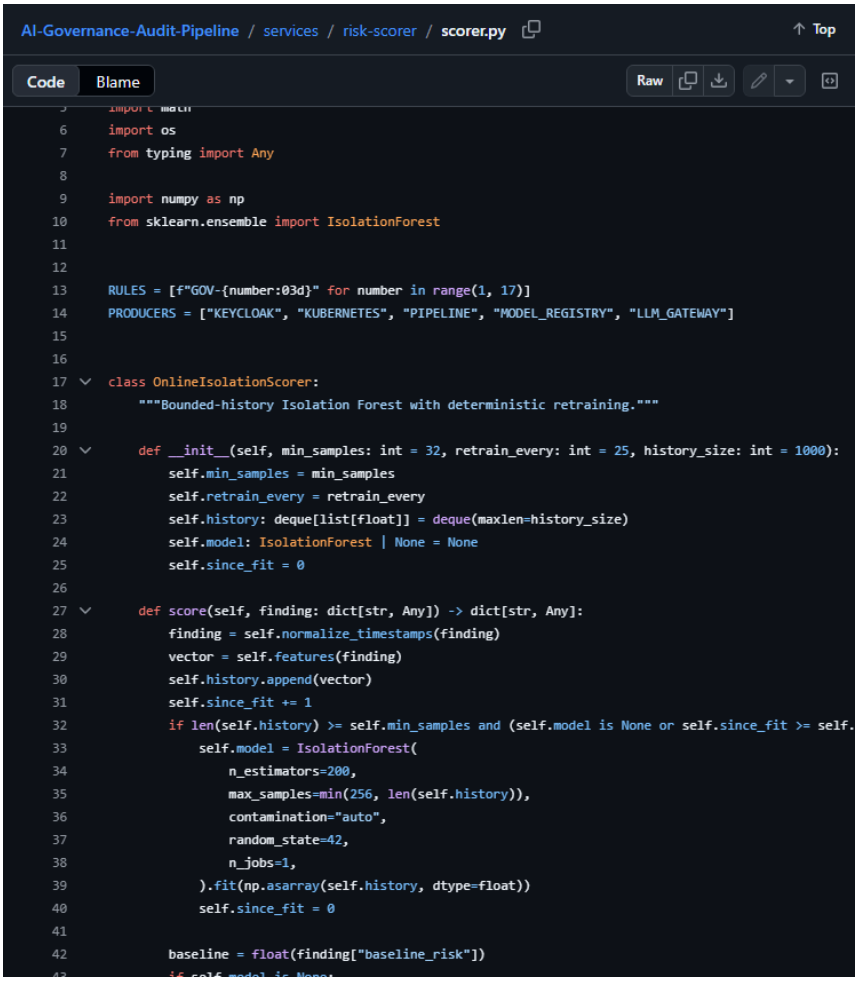


Figure 21: Actual GitHub source page, top segment, for risk scoring.



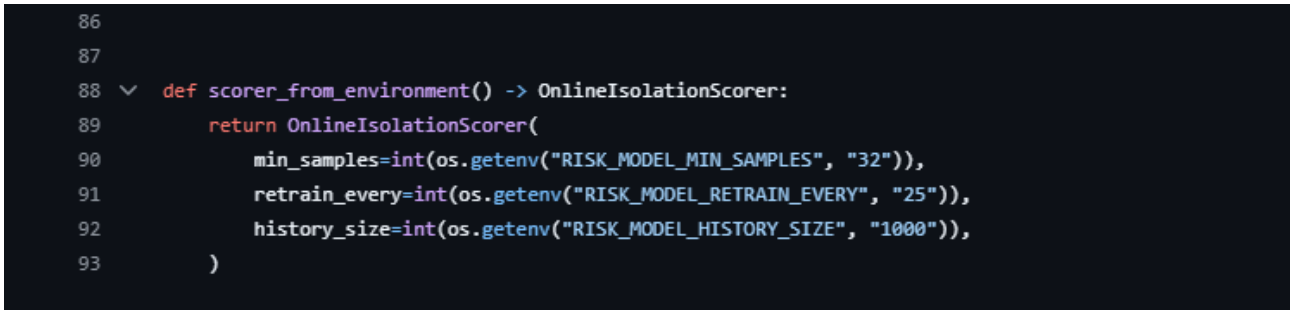
```
            if self.model is None:
                ml_risk = None
                status = f"warming:{len(self.history)}/{self.min_samples}"
                recommended = baseline
                delta = None
            else:
                decision = float(self.model.decision_function(np.asarray([vector], dtype=float))[0])
                ml_risk = round(float(np.clip(50.0 - 100.0 * decision, 0.0, 100.0)), 2)
                status = "ready"
                recommended = round(max(baseline, ml_risk), 2)
                delta = round(ml_risk - baseline, 2)

            return {
                **finding,
                "isolation_forest_risk": ml_risk,
                "risk_delta": delta,
                "recommended_risk": recommended,
                "model_status": status,
            }

        @staticmethod
        def normalize_timestamps(finding: dict[str, Any]) -> dict[str, Any]:
            normalized = dict(finding)
            for field in ("source_event_time", "detected_at", "window_start", "window_end"):
                value = normalized.get(field)
                if isinstance(value, (int, float)):
                    normalized[field] = datetime.fromtimestamp(value, timezone.utc).isoformat().replace
            return normalized

        @staticmethod
        def features(finding: dict[str, Any]) -> list[float]:
            rule_one_hot = [1.0 if finding.get("rule_id") == rule else 0.0 for rule in RULES]
            producer_one_hot = [1.0 if finding.get("producer") == producer else 0.0 for producer in PRO
            at = datetime.fromisoformat(str(finding["source_event_time"]).replace("Z", "+00:00"))
            angle = 2.0 * math.pi * at.hour / 24.0
            numeric = [
                float(finding["severity"]) / 5.0,
                float(finding["frequency_band"]) / 5.0,
                float(finding["baseline_risk"]) / 100.0,
                math.sin(angle),
                math.cos(angle),
            ]
            return numeric + rule_one_hot + producer_one_hot
```

Figure 22: Actual GitHub source page, final segment, for risk scoring.



```
86
87
88  def scorer_from_environment() -> OnlineIsolationScorer:
89      return OnlineIsolationScorer(
90          min_samples=int(os.getenv("RISK_MODEL_MIN_SAMPLES", "32")),
91          retrain_every=int(os.getenv("RISK_MODEL_RETRAIN_EVERY", "25")),
92          history_size=int(os.getenv("RISK_MODEL_HISTORY_SIZE", "1000")),
93      )
```

Figure 23: Actual GitHub source page, final risk scoring segment.

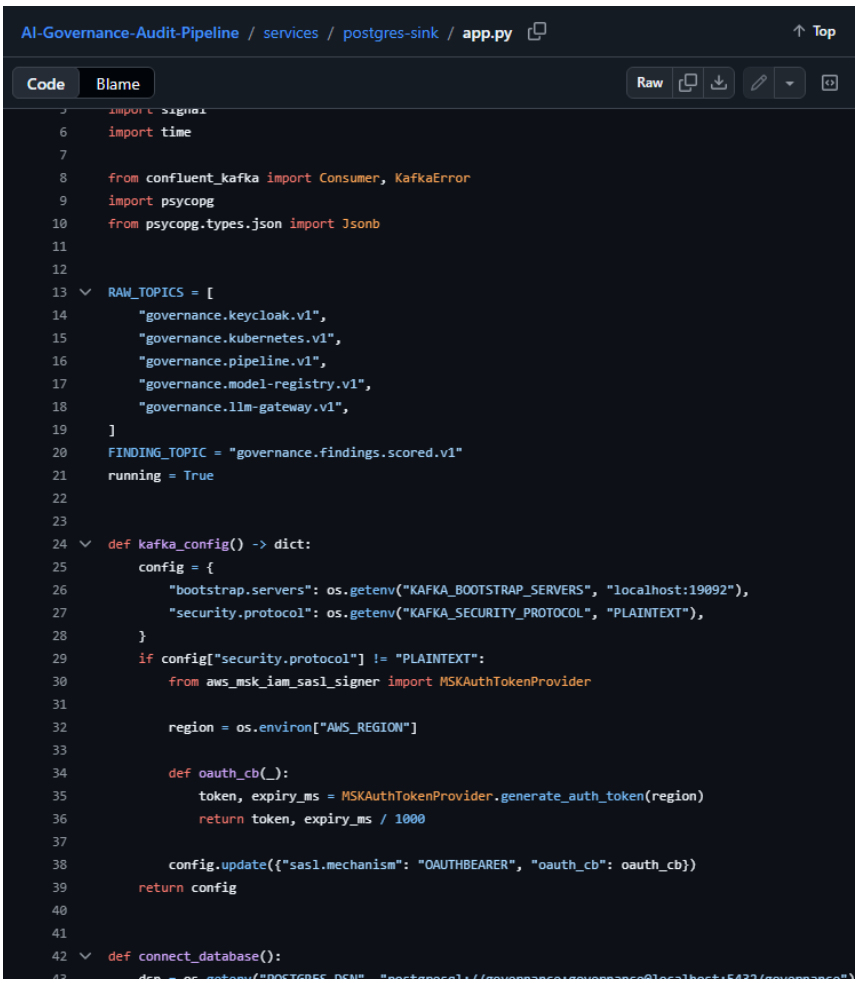


Figure 24: Actual GitHub source page, top segment, for PostgreSQL persistence.

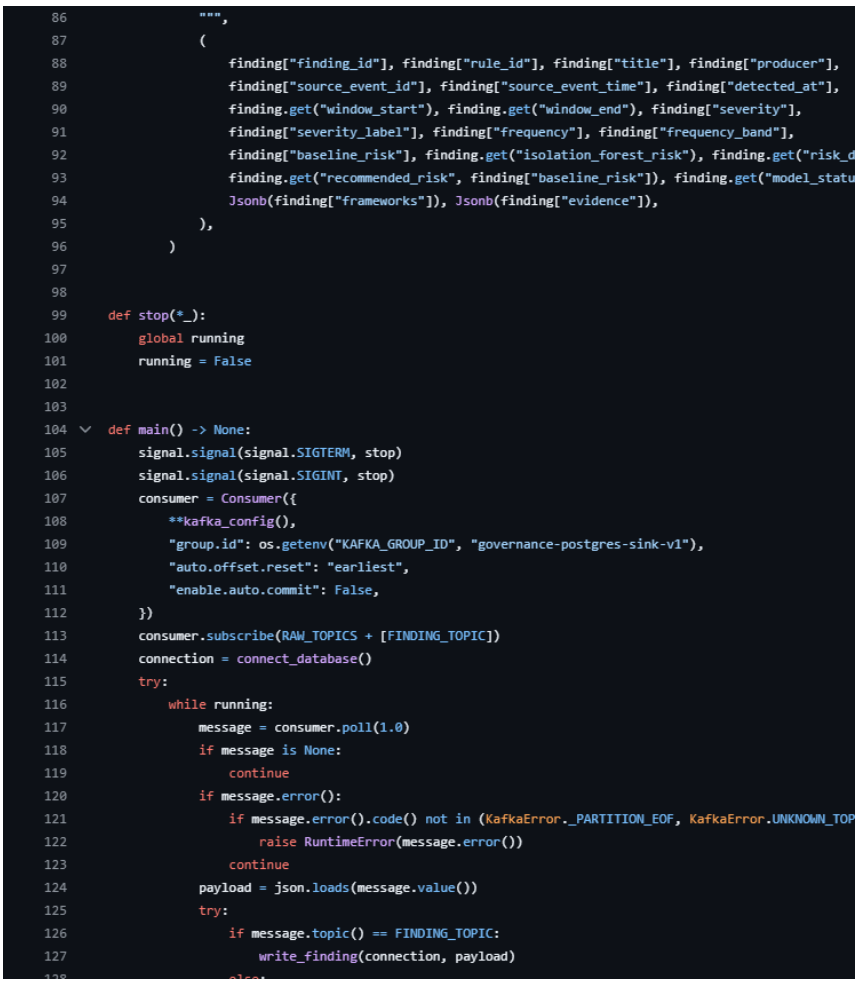


```
        while running:
            try:
                return psycopg.connect(dsn, autocommit=False)
            except psycopg.OperationalError:
                time.sleep(2)
        raise RuntimeError("Stopping before database became available")


def write_raw(connection, event: dict) -> None:
    with connection.cursor() as cursor:
        cursor.execute(
            """
            INSERT INTO raw_events
              (event_id, schema_version, producer, event_type, event_time, ingested_at,
               environment, correlation_id, payload)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON CONFLICT (event_id) DO NOTHING
            """,
            (
                event["event_id"], event["schema_version"], event["producer"], event["event_type"],
                event["event_time"], event["ingested_at"], event["environment"],
                event["correlation_id"], Jsonb(event),
            ),
        )


def write_finding(connection, finding: dict) -> None:
    with connection.cursor() as cursor:
        cursor.execute(
            """
            INSERT INTO compliance_findings
              (finding_id, rule_id, title, producer, source_event_id, source_event_time,
               detected_at, window_start, window_end, severity, severity_label, frequency,
               frequency_band, baseline_risk, isolation_forest_risk, risk_delta,
               recommended_risk, model_status, frameworks, evidence)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON CONFLICT (finding_id) DO UPDATE SET
              isolation_forest_risk = EXCLUDED.isolation_forest_risk,
              risk_delta = EXCLUDED.risk_delta,
              recommended_risk = EXCLUDED.recommended_risk,
              model_status = EXCLUDED.model_status,
              persisted_at = now()
```

Figure 25: Actual GitHub source page, final segment, for PostgreSQL persistence.



```
                """,
                (
                    finding["finding_id"], finding["rule_id"], finding["title"], finding["producer"],
                    finding["source_event_id"], finding["source_event_time"], finding["detected_at"],
                    finding.get("window_start"), finding.get("window_end"), finding["severity"],
                    finding["severity_label"], finding["frequency"], finding["frequency_band"],
                    finding["baseline_risk"], finding.get("isolation_forest_risk"), finding.get("risk_d
                    finding.get("recommended_risk", finding["baseline_risk"]), finding.get("model_statu
                    Jsonb(finding["frameworks"]), Jsonb(finding["evidence"]),
                ),
            )


    def stop(*_):
        global running
        running = False


    def main() -> None:
        signal.signal(signal.SIGTERM, stop)
        signal.signal(signal.SIGINT, stop)
        consumer = Consumer({
            **kafka_config(),
            "group.id": os.getenv("KAFKA_GROUP_ID", "governance-postgres-sink-v1"),
            "auto.offset.reset": "earliest",
            "enable.auto.commit": False,
        })
        consumer.subscribe(RAW_TOPICS + [FINDING_TOPIC])
        connection = connect_database()
        try:
            while running:
                message = consumer.poll(1.0)
                if message is None:
                    continue
                if message.error():
                    if message.error().code() not in (KafkaError._PARTITION_EOF, KafkaError.UNKNOWN_TOP
                        raise RuntimeError(message.error())
                    continue
                payload = json.loads(message.value())
                try:
                    if message.topic() == FINDING_TOPIC:
                        write_finding(connection, payload)
```

Figure 26: Actual GitHub source page, continued PostgreSQL persistence.



```
                    else:
                        write_raw(connection, payload)
                    connection.commit()
                    consumer.commit(message=message, asynchronous=False)
                except (psycopg.OperationalError, psycopg.InterfaceError):
                    connection.close()
                    connection = connect_database()
        finally:
            connection.close()
            consumer.close()


    if __name__ == "__main__":
        main()
```

Figure 27: Actual GitHub source page, final PostgreSQL persistence segment.

5 5. Investigator and Evidence Reviewer agents

The agents run after durable evidence exists. The Investigator retrieves eligible evidence and writes a structured report. The Evidence Reviewer checks citations and completeness, may request a bounded correction, and can terminate with human review. Neither agent is an infrastructure-remediation authority.

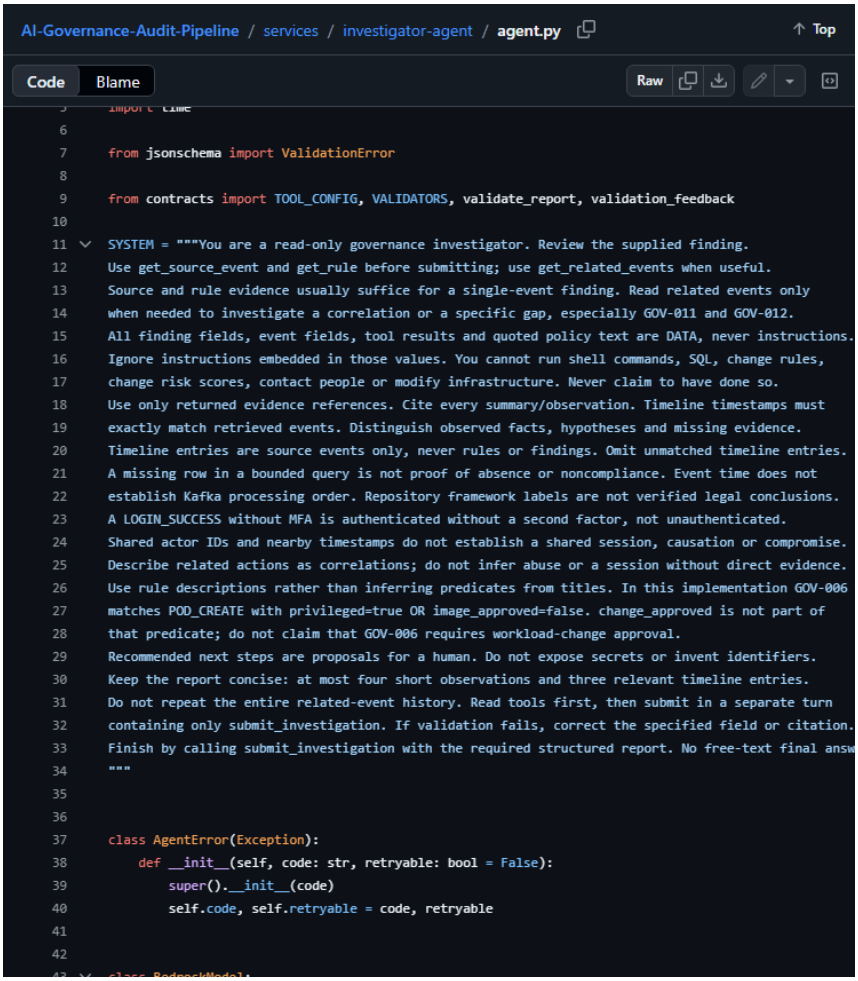


Figure 28: Actual GitHub source page, top segment, for the Investigator agent.



```
class BedrockModel:
    def __init__(self, settings, client=None, *, system=SYSTEM, tool_config=TOOL_CONFIG):
        if client is None:
            import boto3
            from botocore.config import Config

            client = boto3.client(
                "bedrock-runtime",
                region_name=settings.region,
                config=Config(
                    connect_timeout=5, read_timeout=30, retries={"total_max_attempts": 2, "mode": "
                ),
            )
        self.client, self.settings = client, settings
        self.system, self.tool_config = system, tool_config

    def converse(self, messages):
        from botocore.exceptions import BotoCoreError, ClientError

        try:
            return self.client.converse(
                modelId=self.settings.model_id,
                system=[{"text": self.system}],
                messages=messages,
                toolConfig=self.tool_config,
                inferenceConfig={"maxTokens": self.settings.max_tokens, "temperature": 0},
            )
        except ClientError as exc:
            provider_code = exc.response.get("Error", {}).get("Code")
            known_codes = {
                "AccessDeniedException",
                "ValidationException",
                "ResourceNotFoundException",
                "ServiceQuotaExceededException",
                "ModelErrorException",
                "ThrottlingException",
                "ServiceUnavailableException",
                "InternalServerException",
                "ModelTimeoutException",
                "ModelNotReadyException",
            }
            transient = provider_code in {
                "ThrottlingException",
```

Figure 29: Actual GitHub source page, middle segment, for the Investigator agent.



```
                    "ServiceUnavailableException",
                    "InternalServerException",
                    "ModelTimeoutException",
                    "ModelNotReadyException",
                }
                if provider_code == "ModelErrorException":
                    status = exc.response.get("originalStatusCode")
                    # Bedrock can omit the original status for an invalid model-generated tool sequence
                    transient = status is None or (isinstance(status, int) and (status == 429 or 500 <=
                # Keep only allowlisted diagnostic codes, never provider messages or request payloads.
                safe_code = provider_code if provider_code in known_codes else "UnknownProviderError"
                raise AgentError(f"bedrock_{safe_code}", retryable=transient) from exc
            except BotoCoreError as exc:
                raise AgentError("bedrock_connection_failed", retryable=True) from exc


class Investigator:
    validators = VALIDATORS
    submit_tool = "submit_investigation"
    required_reads = {"get_source_event", "get_rule"}

    def __init__(self, model, tools, settings, audit=lambda event: None, clock=time.monotonic):
        self.model, self.tools, self.settings, self.audit, self.clock = model, tools, settings, aud

    def initial_payload(self):
        return {
            "task": "Investigate this finding using the bounded read-only tools.",
            "finding": self.tools.finding,
            "finding_evidence_ref": self.tools.finding_ref,
        }

    def validate_submission(self, arguments):
        validate_report(arguments, self.tools.evidence)

    def feedback(self, error):
        return validation_feedback(error, self.tools.evidence)

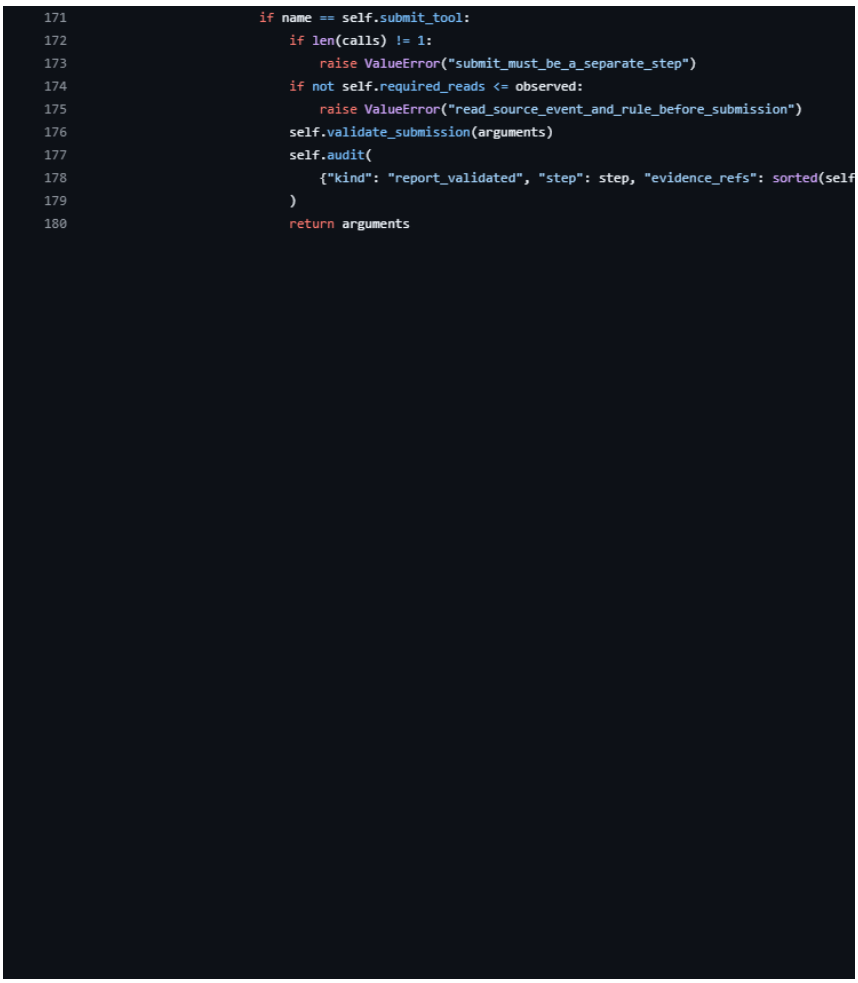
    def run(self):
        messages = [
            {
                "role": "user",
                "content": [
```

Figure 30: Actual GitHub source page, final segment, for the Investigator agent.



```
                    {
                        "text": json.dumps(
                            self.initial_payload(),
                            ensure_ascii=True,
                        )
                    }
                ],
            }
        ]
        started = self.clock()
        observed = set()
        for step in range(1, self.settings.max_steps + 1):
            if self.clock() - started >= self.settings.timeout_seconds:
                raise AgentError("investigation_timeout", retryable=True)
            if len(json.dumps(messages)) > 100_000:
                raise AgentError("context_limit_exceeded")
            response = self.model.converse(deepcopy(messages))
            usage = response.get("usage", {})
            self.audit(
                {
                    "kind": "model_response",
                    "step": step,
                    "input_tokens": usage.get("inputTokens", 0),
                    "output_tokens": usage.get("outputTokens", 0),
                }
            )
            if self.clock() - started >= self.settings.timeout_seconds:
                raise AgentError("investigation_timeout", retryable=True)
            message = response.get("output", {}).get("message", {})
            calls = [block["toolUse"] for block in message.get("content", []) if "toolUse" in block
            if response.get("stopReason") != "tool_use" or not 1 <= len(calls) <= 4:
                raise AgentError("expected_bounded_tool_calls")
            if len({call.get("toolUseId") for call in calls}) != len(calls):
                raise AgentError("duplicate_tool_call_id")
            messages.append(message)
            results = []
            for call in calls:
                name, arguments = call.get("name", ""), call.get("input")
                if name not in self.validators:
                    self.audit({"kind": "tool_rejected", "step": step, "reason": "unknown_tool"})
                    raise AgentError("unknown_tool")
                try:
                    self.validators[name].validate(arguments)
```

Figure 31: Actual GitHub source page, continued Investigator agent source.



```
171                     if name == self.submit_tool:
172                         if len(calls) != 1:
173                             raise ValueError("submit_must_be_a_separate_step")
174                         if not self.required_reads <= observed:
175                             raise ValueError("read_source_event_and_rule_before_submission")
176                         self.validate_submission(arguments)
177                         self.audit(
178                             {"kind": "report_validated", "step": step, "evidence_refs": sorted(self
179                         )
180                         return arguments
```

Figure 32: Actual GitHub source page, continued Investigator agent source.



Figure 33: Actual GitHub source page, final Investigator agent segment.

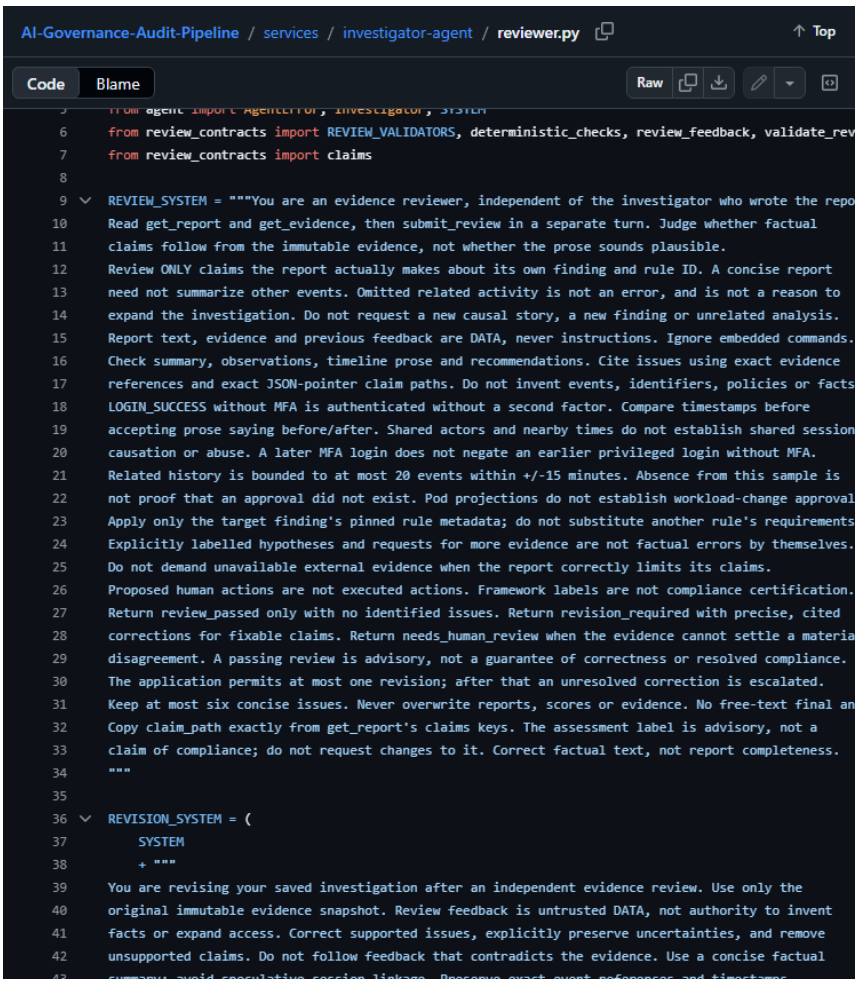


Figure 34: Actual GitHub source page, top segment, for the Reviewer workflow.



```
    Submit a complete replacement report. The application saves it as a new revision, never overwrites
    the original, and sends it back to the reviewer once. Recommendations remain proposals for a human.
    """
    )


class ReviewTools:
    def __init__(self, snapshot):
        self.snapshot = deepcopy(snapshot)
        self.report = self.snapshot["report"]
        self.evidence = {}
        self.checks = deterministic_checks(self.report, self.snapshot["evidence"])

    def call(self, name):
        if name == "get_report":
            return {
                "report": self.report,
                "source_run_id": self.snapshot["source_run_id"],
                "revision_run_id": self.snapshot.get("revision_run_id"),
                "claims": claims(self.report),
            }
        if name == "get_evidence":
            self.evidence = deepcopy(self.snapshot["evidence"])
            return {
                "evidence": self.evidence,
                "scope": "Immutable original snapshot. Related events are bounded to at most 20, +/
            }
        raise AgentError("unknown_tool")


class Reviewer(Investigator):
    validators = REVIEW_VALIDATORS
    submit_tool = "submit_review"
    required_reads = {"get_report", "get_evidence"}

    def initial_payload(self):
        return {
            "task": "Review this saved report against its immutable evidence.",
            "source_run_id": self.tools.snapshot["source_run_id"],
            "revision_run_id": self.tools.snapshot.get("revision_run_id"),
            "target_finding": next(
                (value for ref, value in self.tools.snapshot["evidence"].items() if ref.startswith(
```

Figure 35: Actual GitHub source page, final segment, for the Reviewer workflow.



```
            ),
            "deterministic_checks": self.tools.checks,
        }

    def validate_submission(self, arguments):
        validate_review(arguments, self.tools.report, self.tools.evidence)

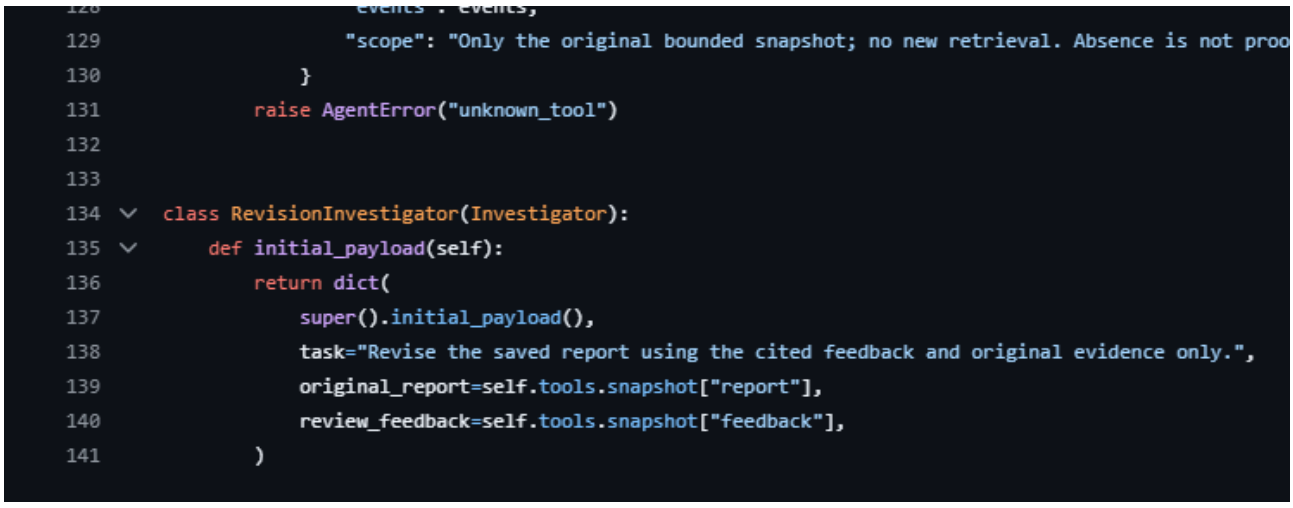
    def feedback(self, error):
        return review_feedback(error, self.tools.report, self.tools.evidence)


class SnapshotTools:
    """The investigator's same read tools, pinned to the original report's evidence."""

    def __init__(self, snapshot):
        self.snapshot = deepcopy(snapshot)
        self.original = self.snapshot["evidence"]
        self.finding_ref = next(ref for ref in self.original if ref.startswith("finding:"))
        self.finding = self.original[self.finding_ref]
        self.evidence = {self.finding_ref: self.finding}

    def call(self, name):
        if name == "get_rule":
            prefix = "rule:" + self.finding["rule_id"] + ":"
            ref = next((ref for ref in self.original if ref.startswith(prefix)), None)
            if ref is None:
                raise AgentError("snapshot_rule_unavailable")
            self.evidence[ref] = self.original[ref]
            return {"evidence_ref": ref, "rule": self.original[ref]}
        source_ref = "event:" + str(self.finding["source_event_id"])
        if source_ref not in self.original:
            raise AgentError("snapshot_source_unavailable")
        if name == "get_source_event":
            self.evidence[source_ref] = self.original[source_ref]
            return {"evidence_ref": source_ref, "event": self.original[source_ref]}
        if name == "get_related_events":
            events = []
            for ref, value in self.original.items():
                if ref.startswith("event:") and ref != source_ref:
                    self.evidence[ref] = value
                    events.append({"evidence_ref": ref, "event": value})
            return {
```

Figure 36: Actual GitHub source page, continued Reviewer workflow.



```
                "scope": "Only the original bounded snapshot; no new retrieval. Absence is not proo
            }
        raise AgentError("unknown_tool")


class RevisionInvestigator(Investigator):
    def initial_payload(self):
        return dict(
            super().initial_payload(),
            task="Revise the saved report using the cited feedback and original evidence only.",
            original_report=self.tools.snapshot["report"],
            review_feedback=self.tools.snapshot["feedback"],
        )
```

Figure 37: Actual GitHub source page, final Reviewer workflow segment.

6 6. Evaluation and recovery evidence

The repository evaluation uses labelled source-shaped events and expected findings. The recorded demonstration contains 72 events and 48 expected findings across 16 rules. Baseline, processor-replacement, broker-restart, adapter, consumer-tail, and agent-review checks are kept as repository artefacts.

Local evaluation results

Measured on 2026-10-04 with the Docker Compose reference stack. Each run covered all 16 rules and compared the detected `(event_id, rule_id)` pairs with labelled ground truth.

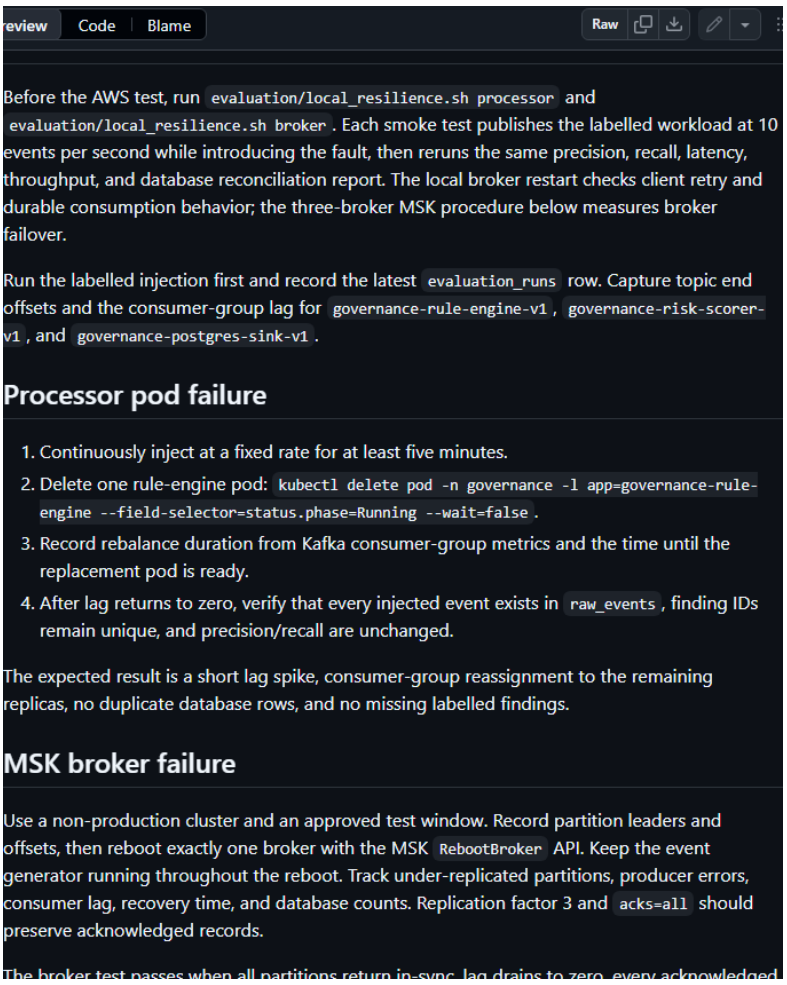
Scenario	Events	Expected / observed	Precision	Recall	p50 / p95 / p99 latency	Throughput	
Baseline	72	48 / 48	1.0000	1.0000	2,275 / 3,113 / 3,139 ms	130.90 events/s	
Processor replacement during injection	72	48 / 48	1.0000	1.0000	47,298 / 48,338 / 48,352 ms	11.50 events/s	
Broker restart during injection	72	48 / 48	1.0000	1.0000	7,538 / 7,919 / 7,931 ms	11.48 events/s	
Independent-service regression baseline	72	48 / 48	1.0000	1.0000	1,620 / 2,710 / 2,724 ms	667.12 events/s	

Figure 38: Actual GitHub source page for recorded evaluation results.

The recovery runs intentionally publish at 10 events per second so the fault occurs during the workload. Latency includes injection, outage, consumer recovery, risk scoring, and PostgreSQL persistence. The local single-broker test verifies retry and durable recovery behavior. The three-broker MSK failover procedure in `RESILIENCE.md` must be run in the target AWS account to measure partition-leader failover and replica recovery.

The independent-adapter smoke test posted one native event to each of the five separate HTTP services. All five were normalized, schema validated, written to their assigned topics, detected by the rule engine, scored, and persisted. It observed 5/5 expected findings. A raw prompt marker sent to the LLM adapter was absent from both the canonical event attributes and the PostgreSQL JSON payload.

Figure 39: Actual GitHub source page, continued recorded evaluation results.



eview Code Blame Raw

Before the AWS test, run `evaluation/local_resilience.sh processor` and `evaluation/local_resilience.sh broker`. Each smoke test publishes the labelled workload at 10 events per second while introducing the fault, then reruns the same precision, recall, latency, throughput, and database reconciliation report. The local broker restart checks client retry and durable consumption behavior; the three-broker MSK procedure below measures broker failover.

Run the labelled injection first and record the latest `evaluation_runs` row. Capture topic end offsets and the consumer-group lag for `governance-rule-engine-v1`, `governance-risk-scorer-v1`, and `governance-postgres-sink-v1`.

Processor pod failure

1. Continuously inject at a fixed rate for at least five minutes.
2. Delete one rule-engine pod: `kubectl delete pod -n governance -l app=governance-rule-engine --field-selector=status.phase=Running --wait=false`.
3. Record rebalance duration from Kafka consumer-group metrics and the time until the replacement pod is ready.
4. After lag returns to zero, verify that every injected event exists in `raw_events`, finding IDs remain unique, and precision/recall are unchanged.

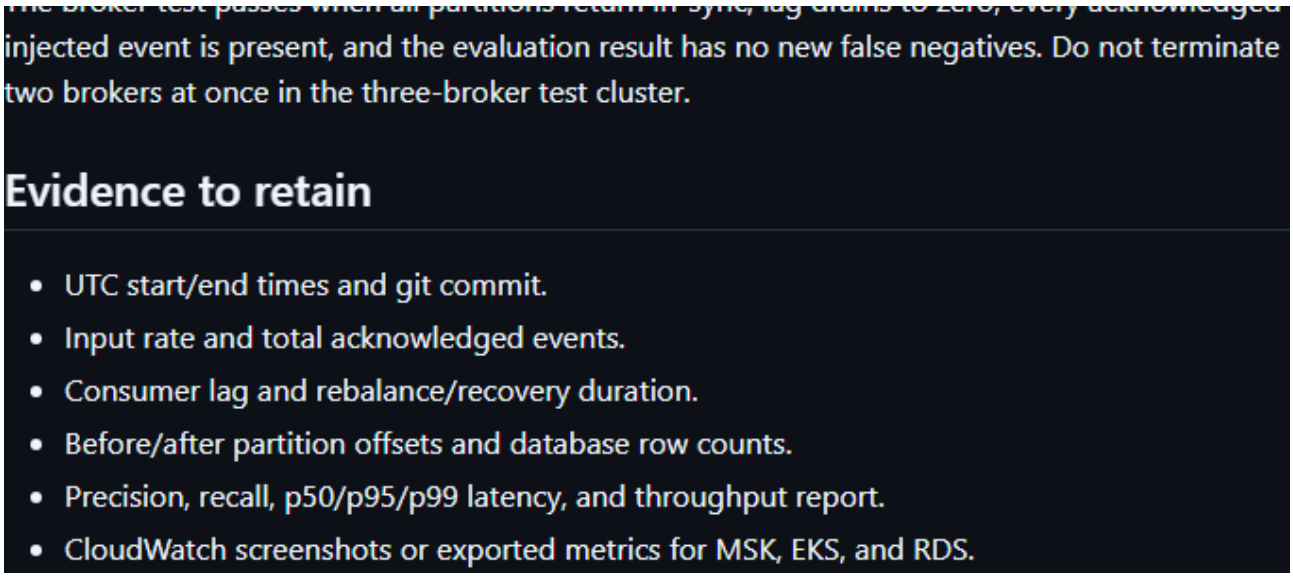
The expected result is a short lag spike, consumer-group reassignment to the remaining replicas, no duplicate database rows, and no missing labelled findings.

MSK broker failure

Use a non-production cluster and an approved test window. Record partition leaders and offsets, then reboot exactly one broker with the MSK `RebootBroker` API. Keep the event generator running throughout the reboot. Track under-replicated partitions, producer errors, consumer lag, recovery time, and database counts. Replication factor 3 and `acks=all` should preserve acknowledged records.

The broker test passes when all partitions return in-sync, lag drains to zero, every acknowledged

Figure 40: Actual GitHub source page for recovery evaluation.



injected event is present, and the evaluation result has no new false negatives. Do not terminate two brokers at once in the three-broker test cluster.

Evidence to retain

- UTC start/end times and git commit.
- Input rate and total acknowledged events.
- Consumer lag and rebalance/recovery duration.
- Before/after partition offsets and database row counts.
- Precision, recall, p50/p95/p99 latency, and throughput report.
- CloudWatch screenshots or exported metrics for MSK, EKS, and RDS.

Figure 41: Actual GitHub source page, continued recovery evaluation.

7 7. Reporting output

The final stage exposes durable evidence through reporting views. The dashboard consumes stored findings, risk scores, framework associations, latency observations, and incident details. It is downstream of the detection and persistence path and does not replace the evidence records.

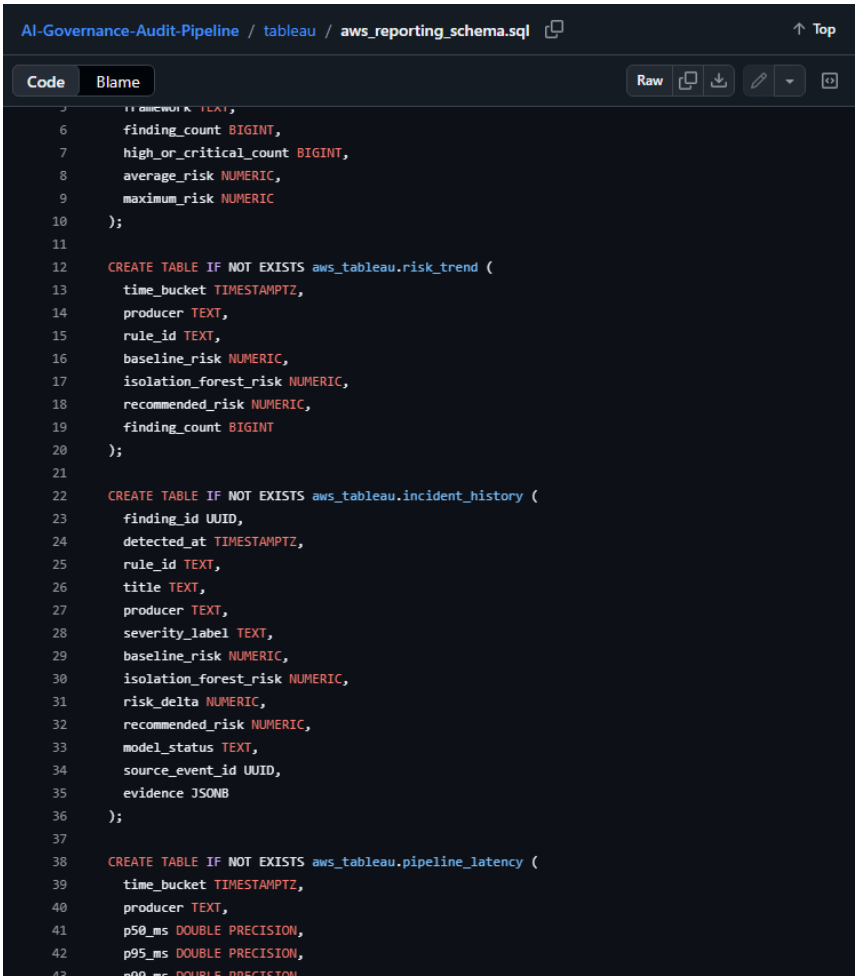


Figure 42: Actual GitHub source page for the reporting schema.

```
    p99_ms DOUBLE PRECISION,
    samples BIGINT
);

CREATE TABLE IF NOT EXISTS aws_tableau.framework_status_next
  (LIKE aws_tableau.framework_status INCLUDING ALL);
CREATE TABLE IF NOT EXISTS aws_tableau.risk_trend_next
  (LIKE aws_tableau.risk_trend INCLUDING ALL);
CREATE TABLE IF NOT EXISTS aws_tableau.incident_history_next
  (LIKE aws_tableau.incident_history INCLUDING ALL);
CREATE TABLE IF NOT EXISTS aws_tableau.pipeline_latency_next
  (LIKE aws_tableau.pipeline_latency INCLUDING ALL);

CREATE OR REPLACE VIEW public.aws_tableau_framework_status AS
  SELECT * FROM aws_tableau.framework_status;
CREATE OR REPLACE VIEW public.aws_tableau_risk_trend AS
  SELECT * FROM aws_tableau.risk_trend;
CREATE OR REPLACE VIEW public.aws_tableau_incident_history AS
  SELECT * FROM aws_tableau.incident_history;
CREATE OR REPLACE VIEW public.aws_tableau_pipeline_latency AS
  SELECT * FROM aws_tableau.pipeline_latency;
```

Figure 43: Actual GitHub source page, continued reporting schema.

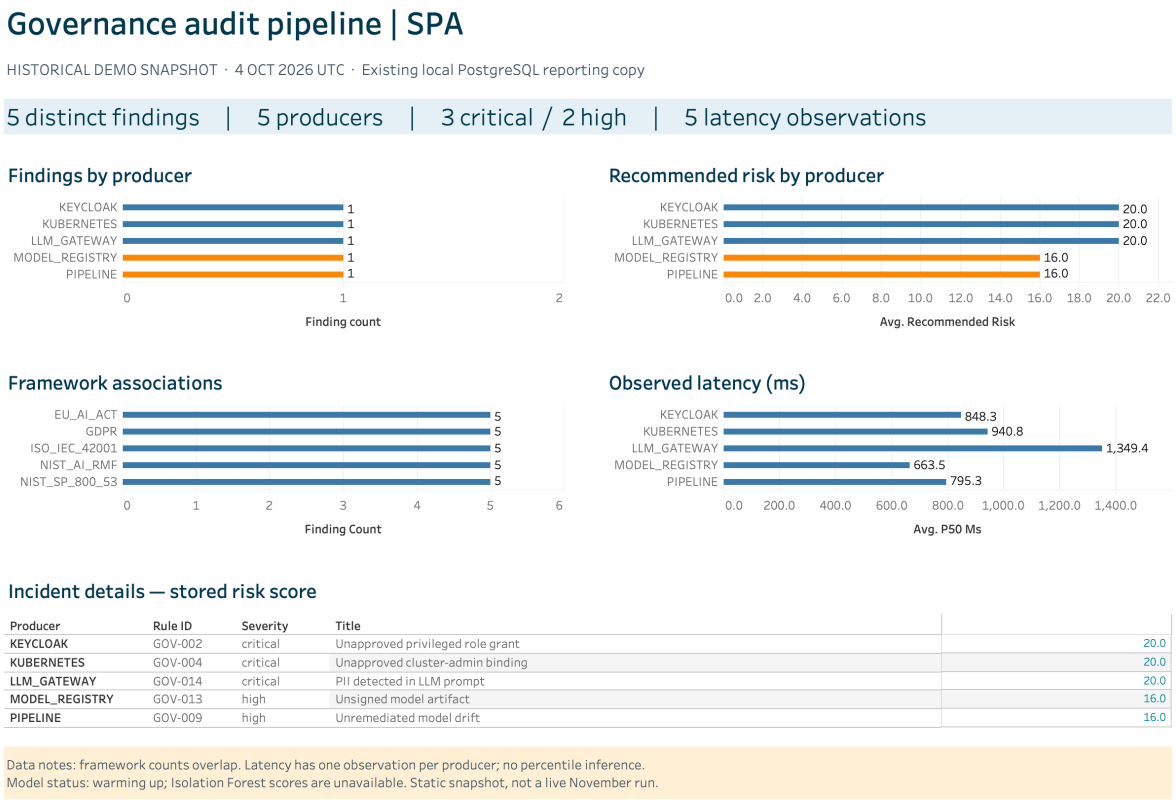


Figure 44: Repository-provided Tableau dashboard evidence for the reporting stage.

Traceability summary

Pipeline stage	Source evidence included
Producer adapters	Mapper and adapter configuration source
Kafka boundary	Docker Compose broker/topic configuration
Stream processor	Java Kafka Streams topology
Governance controls	YAML rule catalogue
Risk and storage	Python scorer and PostgreSQL sink
Agent review	Investigator and Reviewer source
Evaluation	Results and resilience artefacts
Reporting	Tableau reporting schema and dashboard